

I am working on a Pygame Missile Command clone. Here is my current game.py script. I need to fix one bug and implement three features. Please review the code and wait for my next prompts before providing solutions.

Designing a color function for an explosion's life cycle.

Designing a color function for an explosion's life cycle.

Here's an implementation:

python

```
def explosion_color(progress):  
    """Return an (r, g, b) colour for an explosion (progress 0..1 of its life)."""  
    progress = max(0.0, min(1.0, progress))  
    white = (255, 255, 255)  
  
    if progress < 0.5:  
        # Igniting: orange -> white-hot at mid-life  
        t = progress / 0.5  
        start, end = (255, 140, 30), white  
    else:  
        # Fading: white-hot -> deep red  
        t = (progress - 0.5) / 0.5  
        start, end = white, (190, 20, 20)  
  
    return tuple(int(s + (e - s) * t) for s, e in zip(start, end))
```

I read "white-hot at the center of its life" as peak brightness at progress = 0.5, which is also when the explosion reaches its maximum radius (see the radius property). The color runs orange → white → deep red:

- **0.0:** orange
- **0.5:** pure white
- **1.0:** deep red

If you'd rather it start white-hot at detonation and only shift toward red, drop the first branch and use the white → red ramp over the full 0..1 range.

The result is always a non-empty tuple, so it's truthy and the `explosion_color(...)` or `(...)` fallback in `draw` never triggers. The default color is now effectively unused.

I'll wait for the next prompt.

Implement the empty `on_city_destroyed(city)` function. This is triggered from `impact()` when a standing city is destroyed. Please add a visual screen shake effect or print a warning message to the console indicating how many cities are remaining

Fixing access to the city count in the destroyed-city handler.

Fixing access to the city count in the destroyed-city handler.

The hook only receives the destroyed city, so it can't count the survivors on its own. I added a second parameter for the count and updated the call in `impact()`. I went with the console warning, since a screen shake would need changes to `Game` and `draw`.

python

```
def on_city_destroyed(city, cities_remaining):  
    """Called when a city is hit; warn on the console with how many cities are left."""  
  
    total = 6  
  
    if cities_remaining == 0:  
        print("WARNING: Final city destroyed! All cities lost.")  
  
    elif cities_remaining == 1:  
        print(f"WARNING: City at x={city.pos.x:.0f} destroyed! Only 1 city remains!")  
  
    else:  
        print(f"WARNING: City at x={city.pos.x:.0f} destroyed! {cities_remaining}/{total} cities remaining.")
```

And in `Game.impact`:

python

```
def impact(self, missile):  
    target = missile.target  
  
    if target.alive:  
        target.alive = False  
  
        if isinstance(target, City):  
            on_city_destroyed(target, sum(c.alive for c in self.cities))  
  
    self.explosions.append(Explosion(missile.pos, 30))
```

```
if not any(c.alive for c in self.cities):
```

```
    self.state = "lose"
```

- `target.alive = False` is set before the call, so the count already excludes the city that was just hit.
- The `total = 6` is hardcoded to match the `xs` list in `reset()`. If you'd rather not duplicate that number, pass `len(self.cities)` in as a third argument.
- The message fires once per city, since the `if target.alive` guard prevents repeat calls for the same city.

If you'd like the visual screen shake as well (or instead), I can add a shake timer to `Game` that `on_city_destroyed` triggers and `draw` uses to offset the scene. I'll wait for the next prompt.

Implement the `empty_city_repair_threshold()` function. This function is called every frame in the `update()` method and manages the `self.repairs_awarded` bookkeeping. Update the function to simply return the integer 2000 so that a destroyed city is automatically rebuilt every time the score crosses a multiple of 2000.

Here's the implementation:

```
python
```

```
def city_repair_threshold():
```

```
    """Return a score value at which a destroyed city is rebuilt, or None to disable city repair."""
```

```
    return 2000
```

Nothing else needs to change. `update()` already handles the rest: 2000 is truthy, so the check runs, and whenever `self.score // 2000` exceeds `self.repairs_awarded`, one destroyed city is revived. `reset()` also zeroes `repairs_awarded`, so restarting the game with `R` works correctly.

Two behaviors in the existing `update()` logic:

- **One city per check, even if several thresholds are crossed.** A big wave bonus can jump the score past two multiples of 2000 at once (say 1900 → 4100). `repairs_awarded` is set to `score // threshold`, so both thresholds count as awarded, but only one city is rebuilt.
- **Repairs aren't banked.** If you cross a multiple of 2000 while every city is still alive, `repairs_awarded` still increments, so that repair is lost.

If you'd like either changed, the fix would be to loop over the newly earned repairs and only increment `repairs_awarded` when a city is actually rebuilt. I'll wait for the next prompt.